

# LAPORAN REMIDI CLOUD COMPUTING

## MINI CLOUD IMAGE STUDIO

---

### IDENTITAS MAHASISWA

|                       |                            |
|-----------------------|----------------------------|
| <b>Nama Mahasiswa</b> | : Rintan Audina            |
| <b>NIM</b>            | : 32602400035              |
| <b>Kelas</b>          | : TIF 24                   |
| <b>Mata Kuliah</b>    | : Cloud Computing (Remidi) |
| <b>Dosen Pengampu</b> | : Dosen Cloud Computing    |
| <b>Tahun Akademik</b> | : 2026                     |

---

### IDENTITAS MAHASISWA

- Nama: Rintan Audina
- NIM: 32602400035
- Kelas: TIF 24
- Mata Kuliah: Cloud Computing (Remidi)
- Dosen Pengampu: Dosen Cloud Computing
- Tahun Akademik: 2026

---

---

## **BAB 1 - PENDAHULUAN**

---

### **1.1 Latar Belakang**

Perkembangan teknologi cloud computing telah mengubah paradigma arsitektur pengembangan perangkat lunak modern. Pengelolaan aset media seperti gambar pada skala besar memerlukan infrastruktur penyimpanan objek (Object Storage) yang dapat ditingkatkan nilainya secara fleksibel (scalable), efisien, dan andal. Selain itu, indeks metadata dari aset media tersebut perlu dikelola menggunakan basis data NoSQL berkecepatan tinggi agar proses pencarian, filtering, dan manipulasi data dapat berjalan secara instan.

Amazon Web Services (AWS) menyediakan dua layanan utama untuk kebutuhan ini, yaitu Amazon S3 (Simple Storage Service) untuk penyimpanan biner berkas dan Amazon DynamoDB untuk basis data dokumen NoSQL terstruktur. Dalam pengembangan aplikasi tingkat lokal, penggunaan emulasi cloud seperti MinIO dan DynamoDB Local atau LocalStack memungkinkan para pengembang (developers) untuk melakukan simulasi arsitektur cloud AWS secara penuh tanpa biaya operasional.

Oleh karena itu, proyek aplikasi "Mini Cloud Image Studio" ini dibangun menggunakan pemrograman Python, Streamlit, dan Boto3 SDK. Aplikasi ini dirancang untuk memfasilitasi pengunggahan gambar, ekstraksi dan penyimpanan metadata ke DynamoDB, pemrosesan/filter gambar secara interaktif, penyimpanan kembali hasil manipulasi ke S3, serta penayangan galeri gambar secara terintegrasi.

### **1.2 Tujuan**

Tujuan dari pelaksanaan tugas Remidi Cloud Computing ini adalah:

1. Merancang dan membangun aplikasi cloud-native sederhana berbasis Python dan Streamlit.
2. Mengimplementasikan integrasi Boto3 SDK dengan layanan Amazon S3 (MinIO) dan Amazon DynamoDB Local.
3. Membangun fitur manipulasi gambar komprehensif (Resize, Grayscale, Sepia, Invert, Watermark NIM, dan Format Converter) berbasis library Pillow.
4. Mengimplementasikan otomatisasi pembuat (auto-provisioning) S3 Bucket dan DynamoDB Table saat aplikasi diinisialisasi.
5. Menerapkan praktik penanganan error (error handling) yang ramah pengguna (user-friendly) tanpa menampilkan traceback mentah.

### **1.3 Permasalahan**

Permasalahan yang diselesaikan dalam proyek ini meliputi:

1. Bagaimana mengintegrasikan Boto3 SDK agar terhubung ke environment cloud lokal

(MinIO/LocalStack) tanpa memerlukan kredensial AWS asli?

2. Bagaimana mengarsitekturkan pemisahan antara berkas fisik gambar di S3 dengan metadata biner terstruktur di DynamoDB?
3. Bagaimana mengimplementasikan manipulasi gambar interaktif yang responsif pada Streamlit dan menyimpannya secara konsisten kembali ke cloud storage?

---

---

## BAB 2 - PERANCANGAN

---

### 2.1 Arsitektur Sistem

Aplikasi menggunakan arsitektur tiga lapis (three-tier architecture) yang disederhanakan:

Browser Client (Streamlit UI)

?

1. Presentation Layer: Streamlit Web User Interface running pada port 8501.
2. Application & Business Logic Layer: Python modules (services/s3\_service.py, services/dynamodb\_service.py, services/image\_service.py).
3. Infrastructure / Data Layer: MinIO S3 API (Port 9000) dan DynamoDB Local API (Port 8000).

### 2.2 Teknologi Utama

- Python 3.10+: Bahasa pemroses utama.
- Boto3 SDK: Library klien resmi AWS untuk interaksi S3 dan DynamoDB.
- Streamlit: Framework UI web interaktif.
- Pillow (PIL): Engine manipulasi dan transformasi gambar.
- MinIO & DynamoDB Local: Emulator cloud lokal berbasis Docker.

### 2.3 Skema Database DynamoDB

- Nama Tabel: MiniCloudImages
- Partition Key: image\_id (String / HASH)

Attribute schema:

- image\_id (String): ID unik gambar (e.g., IMG-20260820173000-A1B2C3).
- file\_name (String): Nama asli berkas gambar.

- object\_key (String): Key jalur penyimpanan di S3 bucket.
- original\_format (String): Format asli gambar (PNG/JPEG/WEBP).
- processed\_format (String): Format hasil olahan.
- operation (String): Nama operasi manipulasi yang dilakukan.
- width (Number): Lebar gambar dalam piksel.
- height (Number): Tinggi gambar dalam piksel.
- file\_size (Number): Ukuran berkas dalam bytes.
- uploaded\_at (String): Waktu unggah format ISO.
- status (String): Status gambar (Active / Processed).

## **2.4 Struktur Object Storage S3**

- Nama Bucket: mini-cloud-image-studio
- Hierarki Object:
  - originals/: Jalur penyimpan gambar asli.
  - processed/: Jalur penyimpan gambar hasil olahan studio.

---

---

## BAB 3 - IMPLEMENTASI

---

### 3.1 Konfigurasi Cloud Lokal (.env & settings.py)

Pengaturan endpoint dialokasikan melalui variabel lingkungan:

```
AWS_ACCESS_KEY_ID=mock_access_key
```

```
AWS_SECRET_KEY=mock_secret_key
```

### 3.2 Implementasi S3 Service (`s3\_service.py`)

Mengimplementasikan fungsi utama Boto3 S3:

- `check_connection()`: Menguji responsibilitas endpoint S3.
- `ensure_bucket_exists()`: Membuat bucket secara otomatis jika belum ada.
- `upload_image()`: Mengirim byte data ke S3.
- `download_image()`: Mengambil byte data dari S3.
- `delete_image()`: Menghapus object S3.
- `list_images()`: Mengambil daftar objek dalam bucket.

### 3.3 Implementasi DynamoDB Service (`dynamodb\_service.py`)

Mengimplementasikan fungsi CRUD metadata:

- `check_connection()`: Menguji responsibilitas endpoint DynamoDB.
- `ensure_table_exists()`: Membuat tabel MiniCloudImages secara otomatis.
- `save_image_metadata()`: Menyimpan item metadata gambar.
- `get_image_metadata()`: Mengambil item tunggal berdasarkan `image_id`.
- `list_image_metadata()`: Melakukan scan seluruh item metadata.
- `delete_image_metadata()`: Menghapus metadata item berdasarkan `image_id`.

### 3.4 Fitur Pemrosesan Gambar (`image\_service.py`)

1. **Resize**: Mengubah dimensi gambar dengan opsi mempertahankan aspect ratio.

2. Grayscale: Transformasi mode ke warna hitam-putih.
3. Sepia: Penambahan matriks warna warm vintage.
4. Invert: Membalikkan saluran warna RGB.
5. Watermark: Menambahkan teks watermark dan NIM Mahasiswa pada posisi kanan bawah gambar.
6. Format Converter: Mengubah enkripsi biner gambar ke format target (PNG, JPEG, WEBP).

### **3.5 Antarmuka Streamlit (`app.py`)**

UI disusun menjadi 5 bagian utama:

1. Dashboard: Menampilkan metrik utama, status server, dan ringkasan aktivitas.
2. Upload Image: Form unggah berkas, pratinjau, dan tombol upload cloud.
3. Image Studio: Panel kontrol manipulasi gambar interaktif.
4. Gallery & History: Menampilkan kartu galeri berkas lengkap dengan tombol View, Save, dan Delete.
5. Cloud Status: Halaman diagnostik jaringan cloud lokal dan panduan Docker.

---

## BAB 4 - HASIL PENGUJIAN

Pengujian dilakukan untuk memastikan seluruh fungsi aplikasi berjalan sesuai persyaratan tugas:

**Tabel Hasil Pengujian Sistem**

| No | Pengujian               | Skenario Pengujian                                                | Hasil Pengujian                                      | Status       |
|----|-------------------------|-------------------------------------------------------------------|------------------------------------------------------|--------------|
| 1  | Menjalankan Cloud Lokal | Menjalankan container MinIO dan DynamoDB Local via Docker Compose | Service aktif pada port 9000 & 8000                  | BERHASI<br>L |
| 2  | Aplikasi Streamlit      | Menjalankan perintah streamlit run app.py                         | Aplikasi terbuka di browser localhost:8501           | BERHASI<br>L |
| 3  | Auto Bucket S3          | Startup aplikasi mengecek bucket mini-cloud-image-studio          | Bucket terbuat otomatis di MinIO                     | BERHASI<br>L |
| 4  | Auto Table DynamoDB     | Startup aplikasi mengecek tabel MiniCloudImages                   | Tabel terbuat otomatis dengan partition key image_id | BERHASI<br>L |
| 5  | Upload Gambar           | Mengunggah gambar PNG/JPG via form Upload                         | Gambar tersimpan di S3 & preview tampil              | BERHASI<br>L |
| 6  | Metadata DynamoDB       | Pengecekan data setelah upload gambar                             | Item metadata berhasil masuk ke DynamoDB             | BERHASI<br>L |
| 7  | Fitur Resize            | Mengubah ukuran gambar dengan slider dimensi                      | Dimensi gambar berubah sesuai masukan                | BERHASI<br>L |
| 8  | Fitur Grayscale         | Mengaplikasikan filter Grayscale                                  | Gambar berubah menjadi hitam-putih                   | BERHASI<br>L |
| 9  | Fitur Watermark         | Menambahkan watermark teks & NIM Mahasiswa                        | Teks watermark terlukis di pojok kanan bawah         | BERHASI<br>L |
| 10 | Format Converter        | Mengubah format gambar dari PNG ke WEBP                           | Berkas berhasil dikonversi dan disimpan ke S3        | BERHASI<br>L |
| 11 | Download Hasil          | Mengklik tombol Save/Download pada galeri                         | Berkas tersimpan ke direktori komputer lokal         | BERHASI<br>L |
| 12 | Delete Gambar           | Mengklik tombol Delete pada galeri                                | Object di S3 dan metadata di DynamoDB terhapus       | BERHASI<br>L |

---



---

## BAB 5 - KESIMPULAN

---

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Aplikasi Mini Cloud Image Studio telah berhasil dikembangkan menggunakan Python, Streamlit, Boto3 SDK, MinIO (S3), dan DynamoDB Local.
2. Seluruh kebutuhan fungsionalitas mencakup upload gambar, manipulasi gambar (Resize, Grayscale, Sepia, Invert, Watermark NIM, Format Converter), penyimpanan metadata NoSQL, serta fitur galeri (View, Download, Delete) telah 100% terpenuhi.
3. Aplikasi mampu melakukan automated resource initialization untuk bucket S3 dan tabel DynamoDB tanpa memerlukan konfigurasi manual dari pengguna.
4. Penggunaan emulasi cloud lokal memungkinkan aplikasi diuji dan dijalankan secara mandiri di komputer lokal tanpa ketergantungan pada akun AWS berbayar.

---

### ## LAMPIRAN - DAFTAR SCREENSHOT APLIKASI

Berikut adalah daftar dokumentasi tangkapan layar (screenshot) pengujian aplikasi yang tersimpan dalam direktori screenshots/:

1. screenshots/01\_minio\_running.png - Container MinIO / LocalStack Berjalan di Docker Desktop.
2. screenshots/02\_terminal\_running.png - Eksekusi Aplikasi Streamlit (app.py) pada Terminal.
3. screenshots/03\_dashboard.png - Tampilan Dashboard Utama & Cloud Metrics.
4. screenshots/04\_upload\_page.png - Halaman Form Upload Gambar.
5. screenshots/05\_preview\_original.png - Preview Gambar Asli dan Analisis Informasi File.
6. screenshots/06\_image\_processing.png - Panel Kontrol Pemrosesan Gambar (Image Studio).
7. screenshots/07\_grayscale\_resize.png - Hasil Pemrosesan Filter Grayscale dan Resize.
8. screenshots/08\_watermark\_result.png - Hasil Penambahan Custom Watermark dan NIM Mahasiswa.
9. screenshots/09\_format\_converter.png - Fitur Format Converter (PNG / JPEG / WEBP).
10. screenshots/10\_gallery\_history.png - Halaman Galeri Gambar dan Riwayat Aktivitas Cloud.
11. screenshots/11\_s3\_bucket\_objects.png - Tampilan Berkas Objek Gambar pada S3 Bucket (MinIO Console).
12. screenshots/12\_dynamodb\_metadata.png - Tampilan Item Metadata Gambar pada Tabel DynamoDB Local.

13. screenshots/13\_download\_result.png - Proses Download Berkas Hasil Pemrosesan ke Komputer Lokal.

14. screenshots/14\_delete\_image.png - Proses Penghapusan Objek S3 dan Item Metadata DynamoDB.

15. screenshots/15\_github\_repository.png - Struktur Repositori Publik GitHub Proyek.

---

## ## PROMPT AI YANG DIGUNAKAN

Sesuai dengan ketentuan tugas, berikut adalah daftar prompt AI yang secara nyata digunakan selama proses analisis, perancangan, dan pengembangan aplikasi:

### 1. PROMPT ARCHITECTURE:

"Buatlah  
terhubung  
s3\_service."